Rapport SAÉ 2.01-2.02, ressource Graphes

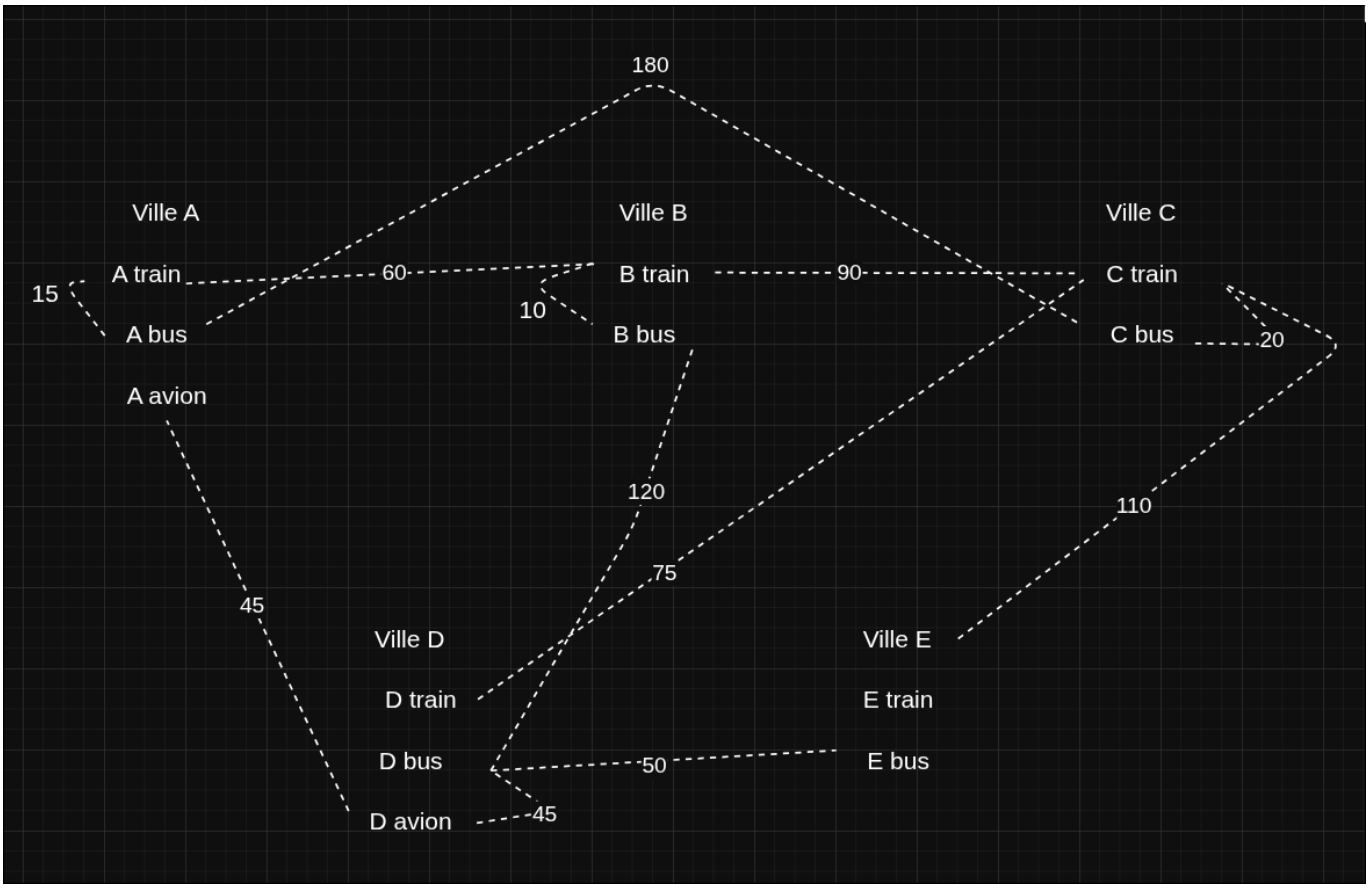
Equipe C2: Erwann Besegher, Alaeddine Oudriss, Jérôme Pardoue, Mehmel Amayas

2 Version 2: Plusieurs moyens de transport

2.1 Exemple

*Nous ne l'avions pas précisé dans le rapport v1 mais les graphes sont dispos au format `.drawio`, c'est l'éditeur que nous avons utilisé pour les réaliser.

Le schéma suivant représente la situation que nous avons modélisée.



Nous avons défini les connexions directes:

Arete	Mode	Duree (min)
A → B	Train	60
B → C	Train	90
A → C	Bus	180
C → D	Train	75
A → D	Avion	45
D → E	Bus	50

Arete	Mode	Duree (min)
C → E	Train	110
B → D	Bus	120

Et les correspondances dans une même ville:

Correspondance	Mode	Duree (min)
A_TRAIN → A_BUS	Bus	15
B_TRAIN → B_BUS	Bus	10
C_BUS → C_TRAIN	Train	20
D_AVION → D_BUS	Bus	45

Scénario d'utilisation : L'utilisateur souhaite voyager de la ville A jusqu'à la ville E. Elle choisit l'avion et le bus. Le critère d'optimisation est le **temps**.

Résultats obtenus : L'algorithme nous renvoie un seul chemin qui correspond au critère que nous avons décidé d'optimiser.

```
javac -cp "lib/*" -d bin src/poo/*.java src/graphes/*.java
java -cp "bin:lib/*" graphes.GraphesV2
--- Démonstration Graphes V2 (Scénario Multimodal) ---
Critère d'optimisation : TEMPS

Recherche d'itinéraires de la ville A vers la ville E :

1) Chemin(Arêtes: [A_AVION -> D_AVION (AVION), [Correspondance] D_AVION ->
D_BUS, D_BUS -> E_BUS (BUS)], Poids: 140.000000)
    -> Temps total : 140 minutes
```

2.2 Modèle pour l'exemple

On passe d'un réseau **unimodal** à un réseau **multimodal**. En V1, le mode de transport était choisi par l'utilisatrice avant la construction du graphe: on gardait seulement les arêtes du mode choisi. En V2, c'est l'algorithme qui combine plusieurs transports si cela permet d'optimiser un critère.

Nous avons donc décomposé les villes avec des moyens de transport comme le bus.

Dans le code, la classe `Ville` de la V1 est remplacée par `SommetEclate`, qui encapsule un couple `(nomVille, modalite)`.

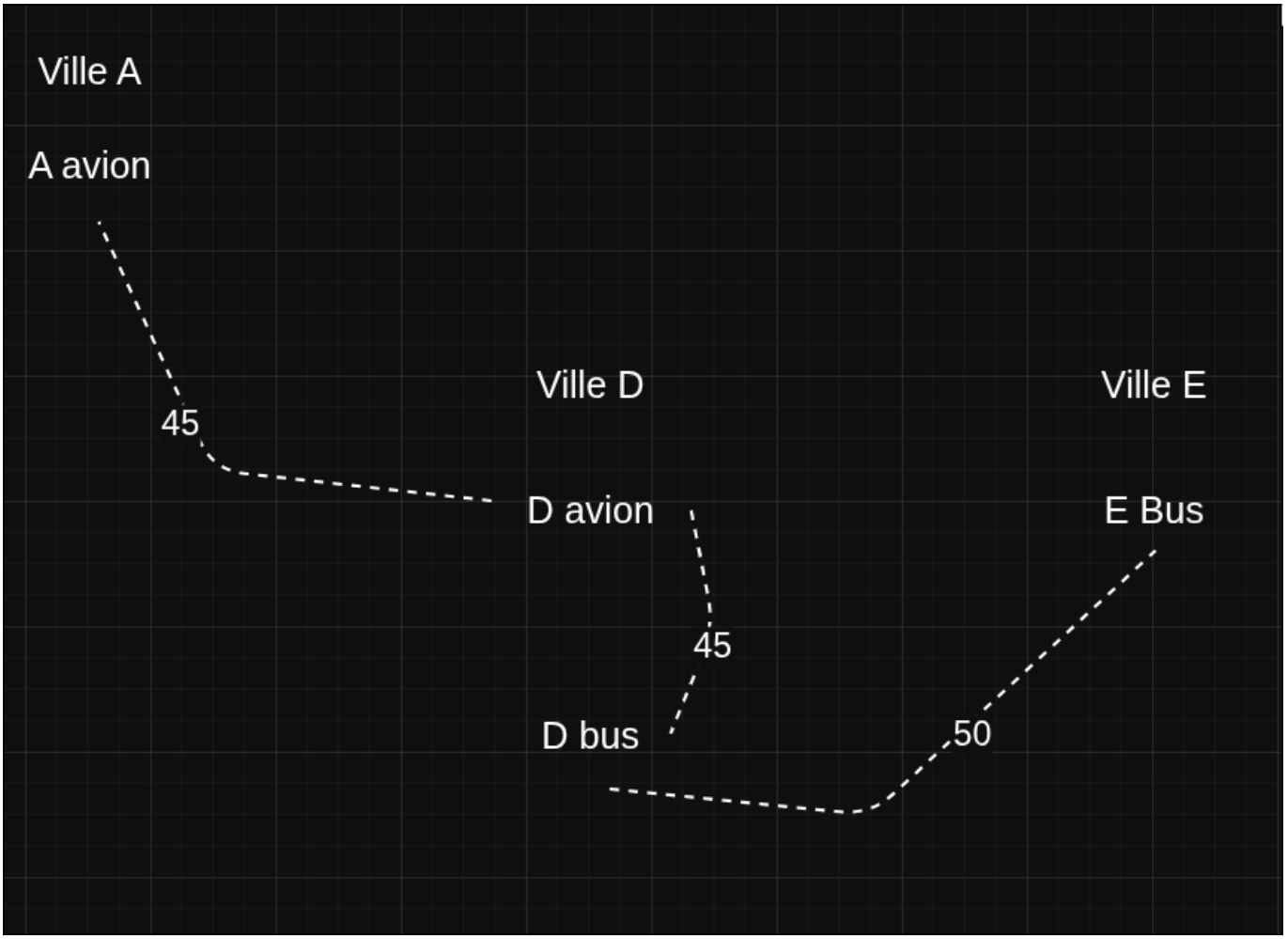
Nous avons deux catégories d'arêtes:

- Les arêtes de transport qui relient deux villes différentes sur la même modalité (A_AVION → D_AVION).

- Les arêtes de correspondance qui relient deux modalités différentes dans même ville (D_AVION → D_BUS).

Dans le code, la classe `Trajet` de la V1 est remplacée par `LienReseau`, qui ajoute un booleen `estCorrespondance` pour distinguer les deux types à l'affichage.

La V2 ajoute des blocs `try-catch` vus en cours de dev. Cela nous permet d'éviter un crash si le point de départ ou d'arrivée demandé ne correspond à rien dans une ville donnée (demander un départ en avion depuis une ville sans aéroport).



2.3 Modélisation pour la Version 2 dans le cas général

- Type de graphe: On garde un multigraphe orienté et valué.
- Les sommets: Chaque ville est séparée en autant de sommets qu'elle dispose de modes de transport disponibles. Un sommet est le couple `(nomVille, modalite)`, implémenté par `SommetEclate`.
- Les arêtes: Deux catégories existent dans le même graphe. Les arêtes de transport relient deux sommets de villes différentes sur la même modalité et les arêtes de correspondance relient deux sommets d'une même ville sur des modalités différentes, et modélisent le coût du transfert.
- L'algorithme: KPCC parcourt le réseau de sommets éclatés. Si emprunter une correspondance et changer de mode réduit le coût global, il le prend en compte. Si l'on souhaitait autoriser le départ depuis n'importe quel mode dans une ville de départ, il faudrait lancer KPCC depuis chaque sommet éclaté de cette ville et conserver les meilleurs résultats globaux.

2.4 Prototype pour la Version 2

Lien du repo gitlab: *https://gitlab.univ-lille.fr/sae2.01-2.02/2026/C2*